07장 파이썬 날아오르기

여러분은 이 책을 다 읽은 후 자신에게 필요한 파이썬 프로그램을 만들어 볼 것이다. 간단한 프로그램이 아니라면 파이썬 표준 라이브러리를 사용해야 하는 순간이 반드시 온다. 그런데 표준 라이브러리 중 일부는 앞에서 배우지 않은 고급 개념을 알아야만 이해할 수 있다. 07장에서는 바로 이 개념들을 알아본다. 파이썬 입문자에게는 조금 어려운 내용이고, 파이썬 코드를 작성하는 데 반드시 알아야 하는 것은 아니므로 건너뛰어도 괜찮다. 하지만 더 깊이 있는 파이썬 프로그램을 작성하고 싶다면 한 번쯤 관심을 두고 익히기를 권한다.

목차

- 07-1 파이썬과 유니코드
- 07-2 클로저와 데코레이터
- 07-3 이터레이터와 제너레이터
- 07-4 파이썬 타입 어노테이션

07-1 파이썬과 유니코드

컴퓨터는 0과 1이라는 값만 인식할 수 있는 기계 장치이다. 그렇다면 컴퓨터는 어떻게 우리가 입력하는 문자를 인식할 수 있는 걸까?

- 최초의 문자 셋, 아스키코드
- 유니코드의 등장
- 유니코드로 문자열 다루기
    - 인코딩하기
    - 디코딩하기
- 입출력과 인코딩
- 소스 코드의 인코딩

최초의 문자 셋, 아스키코드

컴퓨터는 오직 숫자만 이해할 수 있기 때문에 문자를 처리하려면 특별한 방법이 필요하다. 그 방법은 문자와 숫자를 1:1로 매칭시키는 문자 셋(character set)을 만드는 것이다. 예를 들어 숫자 65는 'A', 숫자 66은 'B'와 같이 숫자마다 문자를 매핑해 놓으면 컴퓨터는 해당 숫자를 문자로 대체하여 인식할 수 있다.

초기 컴퓨터 시대에는 각 회사마다 서로 다른 문자 셋을 사용했다. IBM 컴퓨터에서 숫자 65를 'A'로 정의했는데 다른 회사 컴퓨터에서는 같은 숫자 65를 'X'로 정의하는 식이었다. 그 결과 IBM 컴퓨터에서 만든 문서를 다른 컴퓨터에서 열면 완전히 다른 글자가 나타나는 혼란이 벌어졌다.

이 문제를 해결하기 위해 1960년대 미국에서 아스키(ASCII, American Standard Code for Information Interchange)라는 표준을 만들었다. 아스키는 전 세계 모든 컴퓨터가 같은 규칙을 사용하도록 한 최초의 문자 셋 표준이다. 아스키는 총 128개의 문자를 지원했는데, 영문 대소문자, 숫자, 특수문자를 포함하여 영어권 국가에서 사용하는 모든 문자를 처리하는 데는 충분했다.

유니코드의 등장

아스키가 영어권에서는 완벽했지만, 128개 문자로는 한글, 중국어, 일본어, 아랍어 같은 다른 나라의 문자들을 표현할 수 없었다. 한글만 해도 수천 개의 글자가 있고, 중국어는 수만 개의 한자가 존재한다. 그래서 각 나라에서는 자신만의 문자 셋을 만들기 시작했다. 한국에서는 KSC5601을, 일본에서는 JIS를, 중국에서는 GB2312를 만들었다.

하지만 각국이 자신만의 문자 셋을 만들자 더 큰 문제가 생겼다. 한국 컴퓨터에서 만든 문서를 일본 컴퓨터에서 열면 글자가 깨져 보였고, 한글과 중국어를 같은 문서에 함께 쓰는 것도 불가능했다. 인터넷이 발달하면서 이 문제는 더욱 심각해졌다.

1990년대에 등장한 유니코드(Unicode)는 이 모든 문제를 해결했다. 유니코드는 전 세계 모든 문자를 하나의 거대한 표에 정리한 것이다. 현재 유니코드는 100만 개 이상의 문자를 지원할 수 있도록 설계되어 있으며, 한글, 중국어, 일본어, 아랍어는 물론 이집트 상형문자와 이모지까지 포함하고 있다. 유니코드가 전 세계 표준으로 자리 잡으면서 이제 한 문서에 "Hello 안녕하세요 こんにちは 你好"처럼 여러 나라 언어를 자유롭게 섞어 쓸 수 있게 되었다.

유니코드로 문자열 다루기

이제 유니코드로 문자열을 다루는 방법을 알아보자. 파이썬에서 사용하는 문자열은 모두 유니코드 문자열이다.

> *파이썬은 버전 3부터 모든 문자열을 유니코드로 처리한다.*

인코딩하기

다음과 같은 문자열을 살펴보자.

```
>>> a = "Life is too short"
```

유니코드 문자열은 인코딩(encoding) 없이 그대로 파일에 적거나 다른 시스템으로 전송할 수 없다. 왜냐하면 유니코드는 각 문자에 번호를 매긴 약속일 뿐, 그 번호를 실제 바이트로 어떻게 저장할지는 별도로 정해야 하기 때문이다. 파일에 적거나 다른 시스템으로 전송하려면 바이트(byte) 문자열로 변환해야 한다. 이렇게 유니코드 문자열을 바이트 문자열로 바꾸는 것을 '인코딩'이라고 한다. 반대로 바이트 문자열을 유니코드 문자열로 바꾸는 것을 '디코딩'이라고 한다. 같은 문자라도 utf-8, euc-kr 등 서로 다른 방식으로 인코딩할 수 있으므로, 파일을 읽거나 네트워크를 통해 바이트 문자열을 수신할 때는 해당 바이트가 어떤 방식으로 인코딩되었는지 알아야만 디코딩할 수 있다.

유니코드 문자열을 바이트 문자열로 바꾸는 방법은 다음과 같다.

```
>>> a = "Life is too short"
>>> b = a.encode('utf-8')
>>> b
b'Life is too short'
>>> type(b)
<class 'bytes'>
```

유니코드 문자열을 바이트 문자열로 만들 때는 이 예처럼 utf-8과 같은 인코딩 방식을 인수로 넘겨 주어야 한다. 인수를 생략하면 기본값인 utf-8로 동작한다. 결과를 보면 문자열 앞에 `b`가 붙어 있는데, 이것은 해당 문자열이 바이트 문자열임을 나타내는 표시이다. type 함수를 호출해 보면 b 객체는 bytes 클래스의 객체라는 것을 알 수 있다.

이번에는 다음 예제를 살펴보자.

```
>>> a = "한글"
>>> a.encode("ascii")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
UnicodeEncodeError: 'ascii' codec can't encode characters in position 0-1: ordinal not in range(128)
```

이 예에서는 한글이라는 유니코드 문자열을 아스키(ascii) 방식으로 인코딩하려고 시도한다. 하지만 아스키 방식으로는 한글을 표현할 수 없으므로 오류가 발생한다.

"한글"이라는 유니코드 문자열을 바이트 문자열로 변경하는 인코딩 방식에는 여러 가지가 있다. 보통은 utf-8을 사용하지만, 기존 시스템이 euc-kr과 같은 인코딩을 사용한다면 다음과 같이 euc-kr로 인코딩할 수도 있다.

```
>>> a = '한글'
>>> a.encode('euc-kr')
b'\xc7\xd1\xb1\xdb'
>>> a.encode('utf-8')
b'\xed\x95\x9c\xea\xb8\x80'
```

utf-8로 인코딩했을 때와는 다른 바이트 문자열을 출력하는 것을 확인할 수 있다. 여기서 `\x`는 그 뒤의 두 자리가 16진수 값이라는 뜻이다. 예를 들어 `\xc7`은 16진수 c7, 즉 10진수 199에 해당하는 1바이트 값이다.

디코딩하기

이번에는 반대로 인코딩한 바이트 문자열을 유니코드 문자열로 변환하는 디코딩(decoding)을 알아보자. 다음 예제처럼 euc-kr로 인코딩한 바이트 문자열은 euc-kr로만 디코딩해야 한다.

```
>>> a = '한글'
>>> b = a.encode('euc-kr')
>>> b.decode('euc-kr')
'한글'
```

euc-kr로 인코딩한 바이트 문자열을 utf-8로 디코딩하려 한다면 어떻게 될까?

```
>>> b.decode('utf-8')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xc7 in position 0: invalid continuation byte
```

잘못된 인코딩 방식으로 디코딩하려고 하면 이처럼 오류가 발생한다.

지금까지 살펴본 내용을 정리하면 다음과 같다.

```
유니코드 문자열  ---encode('utf-8')--->  바이트 문자열
"한글"  <---decode('utf-8')---   b'\xed\x95\x9c\xea\xb8\x80'
```

- **유니코드**: 전 세계 문자에 번호를 매긴 약속(문자 셋)
- **인코딩**: 유니코드 문자열 → 바이트 문자열 (저장, 전송할 때)
- **디코딩**: 바이트 문자열 → 유니코드 문자열 (읽어올 때)

입출력과 인코딩

인코딩과 관련해서 개발자가 가장 고생하는 부분은 바로 데이터 입출력 관련 작업이다. 문자열과 인코딩에 대한 개념만 확실히 이해하면 어렵지 않지만, 이를 이해하지 못하고 무작정 인코딩, 디코딩을 사용하면 다중 인코딩되거나 문자열이 꼬여 버리는 불상사가 발생하기도 한다.

핵심 원칙은 간단하다. 프로그램 내부에서는 항상 유니코드 문자열을 사용하고, 외부와 데이터를 주고받을 때만 인코딩/디코딩하는 것이다. 파일을 읽거나 네트워크를 통해 데이터를 주고받을 때 추천하는 방법은 다음과 같다.

1. 입력으로 받은 바이트 문자열은 되도록 빨리 유니코드 문자열로 디코딩한다.
2. 함수나 클래스 등에서는 유니코드 문자열만 사용한다.
3. 입력에 대한 결과를 전송하는 마지막 부분에서만 유니코드 문자열을 바이트 문자열로 인코딩해서 반환한다.

이 규칙을 지킨다면 인코딩과 관련해서 큰 어려움은 없을 것이다.

다음은 euc-kr 방식으로 작성한 파일을 읽고 변경하여 저장하는 예제이다.

```
# euc_kr.py
# 1. euc-kr로 작성된 파일 읽기
with open('euc_kr.txt', encoding='euc-kr') as f:
    data = f.read()  # 유니코드 문자열

# 2. unicode 문자열로 프로그램 수행하기
data = data + "\n" + "추가 문자열"

# 3. euc-kr로 수정된 문자열 저장하기
with open('euc_kr.txt', encoding='euc-kr', mode='w') as f:
    f.write(data)
```

open() 함수에 encoding을 지정하면 해당 인코딩으로 파일을 읽을 수 있다. 이때 읽은 문자열은 유니코드 문자열이 된다. 파일을 만들 때도 마찬가지로 encoding을 지정할 수 있다. encoding 항목을 생략하면 기본값으로 utf-8이 지정된다.

소스 코드의 인코딩

파이썬 셸이 아닌 편집기로 코딩할 때는 소스 코드의 인코딩이 매우 중요하다. 소스 코드의 인코딩이란 소스 코드 파일이 어떤 방식으로 인코딩되었는지를 뜻한다.

> *현재 대부분의 편집기와 파이썬 3는 utf-8을 기본으로 사용하므로 인코딩을 별도로 선언할 일이 거의 없다. 다만 euc-kr 같은 다른 인코딩으로 저장된 파일을 다루는 특수한 경우에는 알아 두어야 한다.*

소스 코드도 파일이므로 인코딩 타입이 반드시 필요하다. 파이썬은 소스 코드의 인코딩을 명시하려면 소스 코드 가장 위에 다음과 같은 문장을 넣어야 한다.

```
# -*- coding: utf-8 -*-
```

> *파이썬 3.0부터는 utf-8이 기본값이므로 utf-8로 인코딩한 소스 코드라면 이 문장은 생략해도 된다.*

소스 코드를 utf-8로 인코딩한 파일이라면 이렇게 작성하면 되고 euc-kr로 인코딩했다면 다음과 같이 작성해야 한다.

```
# -*- coding: euc-kr -*-
```

소스 코드는 euc-kr로 인코딩했는데 파일 위에 utf-8로 명시했다면 문자열 처리 부분에서 인코딩 관련 오류가 발생할 수 있다.

07-2 클로저와 데코레이터

데코레이터를 이해하려면 먼저 클로저를 알아야 한다. 클로저를 먼저 알아보고 나서 데코레이터를 살펴보자.

- 클로저란?
- 데코레이터란?

클로저란?

클로저란 **외부 함수의 변수를 기억하는 내부 함수**를 말한다.

예를 들어, 함수 A 안에 함수 B가 있고, 함수 B가 함수 A의 변수를 사용한다고 하자. 보통은 함수 A가 끝나면 함수 A의 변수도 사라진다. 하지만 클로저에서는 함수 A가 끝나도 함수 B가 함수 A의 변수를 계속 사용할 수 있다. 마치 함수 A의 변수가 함수 B 안에 저장되어 있는 것처럼 말이다.

이해하기 어려울 수 있지만, 다음 예제를 보면 클로저가 왜 유용한지 알 수 있다.

어떤 수에 항상 3을 곱해 반환하는 함수를 생각해 보자. 아마도 다음과 같은 함수를 만들 수 있을 것이다.

```
def mul3(n):
    return n * 3
```

mul3() 함수는 입력으로 받은 수 n에 항상 3을 곱하여 반환한다. 이번에는 항상 5를 곱하여 반환하는 함수를 생각해 보자.

```
def mul5(n):
    return n * 5
```

이처럼 mul5() 함수를 만들 수 있을 것이다. 하지만 필요할 때마다 mul6(), mul7(), mul8(), …과 같은 함수를 만드는 것은 매우 비효율적이다. 이 문제를 효율적으로 해결하려면 다음과 같이 클래스를 사용하면 된다.

```
# closure.py
class Mul:
    def __init__(self, m):
        self.m = m

    def mul(self, n):
        return self.m * n

if __name__ == "__main__":
    mul3 = Mul(3)
    mul5 = Mul(5)

    print(mul3.mul(10))  # 30 출력
    print(mul5.mul(10))  # 50 출력
```

클래스를 이용해 특정 값을 미리 설정하고 그다음부터 mul() 메서드를 사용하면 원하는 형태로 호출할 수 있다. `__call__` 메서드를 이용하면 다음과 같이 개선할 수도 있다.

```
# closure.py
class Mul:
    def __init__(self, m):
        self.m = m

    [[MARK]]def __call__(self, n):[[/MARK]]
        return self.m * n

if __name__ == "__main__":
    mul3 = Mul(3)
    mul5 = Mul(5)

    [[MARK]]print(mul3(10))  # 30 출력
    print(mul5(10))  # 50 출력[[/MARK]]
```

mul() 함수의 이름을 `__call__` 로 바꾸었다. `__call__` 은 객체를 함수처럼 호출할 수 있게 해주는 특수 메서드이다. 즉, `mul3(10)` 처럼 객체 뒤에 괄호를 붙여 호출하면 파이썬이 자동으로 `__call__` 메서드를 실행한다. 이전 코드에서는 `mul3.mul(10)` 처럼 메서드 이름을 명시해야 했지만, `__call__` 을 사용하면 `mul3(10)` 처럼 간결하게 호출할 수 있다.

이렇게 클래스로 만드는 방법이 일반적이긴 하지만, 더 간편한 방법이 있다. 다음 함수를 살펴보자.

```
# wrapper.py
def mul(m):
    def wrapper(n):
        return m * n
    return wrapper

if __name__ == "__main__":
    mul3 = mul(3)
    mul5 = mul(5)

    print(mul3(10))  # 30 출력
    print(mul5(10))  # 50 출력
```

외부 함수 mul 안에 내부 함수 wrapper를 구현했다. 그리고 외부 함수는 내부 함수 wrapper를 반환한다. 함수가 함수를 반환하는 것이 생소할 수 있겠지만, 파이썬에서 함수는 숫자나 문자열처럼 변수에 저장하거나 다른 함수의 입력으로 전달하거나 반환할 수 있는 객체이므로 이런 사용이 가능하다.

클로저의 핵심은 바로 여기에 있다! `mul(3)`을 호출하면 외부 함수 mul은 종료되지만, 반환된 wrapper 함수는 여전히 `m=3` 값을 기억하고 있다. 이후 `mul3(10)`을 호출하면 wrapper 함수가 기억하고 있던 `m=3` 값을 사용해서 `3 * 10 = 30`을 계산한다.

즉, 클로저는 **함수가 생성될 때의 환경(변수 값)을 기억하는 특별한 함수**라고 할 수 있다. 여기서 반환된 wrapper 함수가 바로 클로저이다. 그리고 mul 함수처럼 클로저를 만들어내는 함수를 클로저 팩토리(closure factory) 함수라고 한다. 공장(factory)에서 제품을 찍어내듯이, 클로저 팩토리 함수는 호출할 때마다 새로운 클로저를 만들어 낸다.

데코레이터란?

데코레이터(decorator)는 클로저를 활용하여 기존 함수를 수정하지 않고 기능을 덧붙이는 기법이다. 어떤 경우에 유용한지 예제를 통해 알아보자.

다음은 "함수가 실행됩니다."라는 문자열을 출력하는 myfunc 함수이다.

```
def myfunc():
    print("함수가 실행됩니다.")
```

이 함수의 실행 시간을 측정해야 한다면 어떻게 해야 할까? 함수 실행 시간은 함수가 시작하는 순간의 시간과 종료되는 순간의 시간 차이를 구하면 알 수 있다. 따라서 다음과 같이 코드를 수정하면 함수의 실행 시간을 측정할 수 있다.

```
import time

def myfunc():
    start = time.time()
    print("함수가 실행됩니다.")
    end = time.time()
    print("함수 수행시간: %f 초" % (end-start))

myfunc()
```

하지만 실행 시간을 측정해야 하는 함수가 myfunc 말고도 많다면 이런 코드를 모든 함수에 적용하는 것은 너무 비효율적이다. 이때 클로저를 이용하면 좀 더 효율적인 방법을 찾을 수 있다.

```
# decorator.py
import time

def elapsed(original_func):   # 기존 함수를 인수로 받는다.
    def wrapper():
        start = time.time()
        result = original_func()    # 기존 함수를 수행한다.
        end = time.time()
        print("함수 수행시간: %f 초" % (end - start))  # 기존 함수의 수행시간을 출력한다.
        return result  # 기존 함수의 수행 결과를 반환한다.
    return wrapper

def myfunc():
    print("함수가 실행됩니다.")

decorated_myfunc = elapsed(myfunc)
decorated_myfunc()
```

elapsed 함수로 클로저를 만들었다. 이 함수는 함수를 인수로 받는다. 파이썬은 함수도 객체이므로 함수 자체를 인수로 전달할 수 있다.

이제 `decorated_myfunc = elapsed(myfunc)` 로 생성한 decorated_myfunc를 decorated_myfunc()로 실행하면 실제로는 elapsed 함수 내부의 wrapper 함수가 실행되고, 이 함수는 전달받은 myfunc 함수를 실행하면서 실행 시간을 함께 출력한다.

클로저를 이용하면 기존 함수에 기능을 덧붙이기가 매우 편리하다. 이렇게 기존 함수를 바꾸지 않고 기능을 추가할 수 있게 만드는 elapsed 함수와 같은 클로저를 데코레이터(decorator)라고 한다.

> *'decorate'는 '꾸미다, 장식하다'라는 뜻이므로 데코레이터를 함수를 꾸미는 함수라고 생각해도 된다.*

이 코드를 실행하면 다음과 같은 결과가 출력된다.

```
함수가 실행됩니다.
함수 수행시간: 0.000029 초
```

파이썬 데코레이터는 다음처럼 @ 문자를 이용해 함수 위에 적용하여 사용할 수도 있다.

```
# decorator.py
import time

def elapsed(original_func):   # 기존 함수를 인수로 받는다.
    def wrapper():
        start = time.time()
        result = original_func()    # 기존 함수를 수행한다.
        end = time.time()
        print("함수 수행시간: %f 초" % (end - start))  # 기존 함수의 수행시간을 출력한다.
        return result  # 기존 함수의 수행 결과를 반환한다.
    return wrapper

[[MARK]]@elapsed[[/MARK]]
def myfunc():
    print("함수가 실행됩니다.")

# decorated_myfunc = elapsed(myfunc)  # @elapsed 데코레이터로 인해 더이상 필요하지 않다.
# decorated_myfunc()

myfunc()
```

myfunc 함수 바로 위에 `@elapsed` 라는 데코레이터를 추가했다. 파이썬은 함수 위에 `@함수명` 이 있으면 데코레이터 함수로 인식한다. 따라서 이제 myfunc 함수는 elapsed 데코레이터를 통해 수행될 것이다.

프로그램을 실행해 보면 마찬가지 결과가 출력되는 것을 확인할 수 있다.

```
함수가 실행됩니다.
함수 수행시간: 0.000029 초
```

이번에는 myfunc 함수를 다음과 같이 변경해 보자.

```
# decorator2.py

... 생략 ...

@elapsed
def myfunc(msg):
    print("'%s'을 출력합니다." % msg)

myfunc("You need python")  # 출력할 메시지를 myfunc 파라미터로 전달한다.
```

문자열을 입력받아 출력하도록 myfunc 함수를 수정했다. 하지만 이렇게 코드를 수정하고 실행하면 다음과 같은 오류가 발생한다.

```
Traceback (most recent call last):
  File ... 생략 ...
    myfunc("You need python")
TypeError: wrapper() takes 0 positional arguments but 1 was given
```

myfunc 함수는 입력 인수가 필요하지만, elapsed 함수 안의 wrapper 함수는 전달받은 myfunc 함수를 입력 인수 없이 호출해 오류가 발생한 것이다. 그러므로 데코레이터 함수는 기존 함수의 입력 인수에 상관없이 동작하도록 만들어야 한다. 데코레이터는 기존 함수가 어떤 입력 인수를 취할지 알 수 없기 때문이다. 이렇게 전달받아야 하는 기존 함수의 입력 인수를 알 수 없는 경우에는 `*args`와 `**kwargs` 매개변수를 이용하면 된다. `*args`는 입력 인수의 개수에 상관없이 모든 입력을 받을 수 있고, `**kwargs`는 `키=값` 형태의 모든 입력을 받을 수 있는 매개변수이다.

다음과 같이 코드를 수정하자.

```
# decorator2.py
import time

def elapsed(original_func):   # 기존 함수를 인수로 받는다.
    def wrapper([[MARK]]*args, **kwargs[[/MARK]]):   # *args, **kwargs 매개변수 추가
        start = time.time()
        result = original_func([[MARK]]*args, **kwargs[[/MARK]])  # 전달받은 *args, **kwargs를 입력파라미터로 기존함수 수행
        end = time.time()
        print("함수 수행시간: %f 초" % (end - start))  # 수행시간을 출력한다.
        return result  # 함수의 결과를 반환한다.
    return wrapper

@elapsed
def myfunc(msg):
    """ 데코레이터 확인 함수 """
    print("'%s'을 출력합니다." % msg)

myfunc("You need python")
```

wrapper() 함수의 매개변수로 `*args` 와 `**kwargs` 를 추가하고 기존 함수 실행 시 `*args` 와 `**kwargs` 를 인수로 전달하여 호출하게 했다. 이제 프로그램을 실행하면 오류 없이 다음과 같은 결과를 출력한다.

```
'You need python'을 출력합니다.
함수 수행시간: 0.000027 초
```

지금까지 클로저와 데코레이터에 대해 알아보았다.

점프 투 파이썬

`*args` 와 `**kwargs`

`*args` 는 모든 입력 인수를 튜플로 변환하는 매개변수, `**kwargs` 는 모든 '키=값' 형태의 입력 인수를 딕셔너리로 변환하는 매개변수이다. 다음과 같은 형태의 호출을 살펴보자.

```
>>> func(1, 2, 3, name='foo', age=3)
```

func 함수가 입력 인수의 개수와 형태에 상관없이 모든 입력을 처리하려면 어떻게 해야 할까?

```
>>> def func([[MARK]]*args, **kwargs[[/MARK]]):
...     print(args)
...     print(kwargs)
...
>>> func(1, 2, 3, name='foo', age=3)
(1, 2, 3)
{'age': 3, 'name': 'foo'}
```

이처럼 func 함수에 `*args` , `**kwargs` 라는 매개변수를 지정하면 다양한 입력 인수를 모두 처리할 수 있다. 이렇게 하면 1, 2, 3 같은 일반 입력은 args 튜플, `name = 'foo'` 와 같은 '키=값' 형태의 입력은 kwargs 딕셔너리로 저장한다.

07-3 이터레이터와 제너레이터

파이썬으로 대량의 데이터를 처리하다 보면 메모리를 효율적으로 사용하고, 필요한 만큼만 데이터를 가져오는 방법이 필요하다. 이터레이터와 제너레이터는 이런 문제를 해결하는 파이썬의 핵심 도구이다.

다음은 리스트의 간단한 사용법이다.

```
for a in [1, 2, 3]:
    print(a)
```

리스트 [1, 2, 3]을 for 문으로 차례대로 하나씩 출력하는 예제이다. 이렇게 for 문과 같은 반복 구문에 사용할 수 있는 리스트와 같은 객체를 '반복 가능(iterable) 객체'라고 한다.

- 이터레이터란?
- 이터레이터 만들기
- 제너레이터란?
- 제너레이터 표현식
- 클래스 이터레이터와 제너레이터
- 제너레이터 활용하기

이터레이터란?

그렇다면 이터레이터(iterator)란 무엇일까? 이터레이터는 **next() 함수로 값을 하나씩 꺼낼 수 있는 객체**이다. 모든 값을 꺼내면 StopIteration 예외가 발생한다.

리스트는 반복 가능(iterable)하다는 것을 이미 알아보았다. 그렇다면 리스트는 이터레이터일까? 다음과 같이 확인해 보자.

```
>>> a = [1, 2, 3]
>>> next(a)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'list' object is not an iterator
```

리스트 a로 next 함수를 호출했더니 리스트는 이터레이터가 아니라는 오류가 발생한다. 즉, 반복 가능하다고 해서 이터레이터는 아니다.

비유하면 책(iterable)은 읽을 수 있는 대상이지만, 실제로 페이지를 한 장씩 넘기며 읽으려면 책갈피(iterator)가 필요한 것과 같다. 리스트는 데이터를 담고 있는 책이고, 이터레이터는 그 책에서 데이터를 하나씩 꺼내 주는 책갈피 역할을 한다.

반복 가능한 객체는 iter 함수를 이용해 이터레이터로 만들 수 있다.

```
>>> a = [1, 2, 3]
>>> ia = [[MARK]]iter(a)[[/MARK]]
>>> type(ia)
<class 'list_iterator'>
```

이제 리스트를 이터레이터로 변경했으므로 next 함수를 호출해 보자.

```
>>> next(ia)
1
>>> next(ia)
2
>>> next(ia)
3
>>> next(ia)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
StopIteration
```

next 함수를 호출할 때마다 이터레이터의 요소를 차례대로 반환한다. 더 이상 반환할 값이 없으면 StopIteration 예외가 발생한다.

이터레이터의 값을 가져오는 가장 일반적인 방법은 for 문을 이용하는 것이다.

```
>>> a = [1, 2, 3]
>>> ia = iter(a)
>>> for i in ia:
...     print(i)
...
1
2
3
```

for 문을 이용하면 next 함수를 직접 호출하거나 StopIteration 예외를 처리할 필요가 없다. for 문이 이 과정을 자동으로 처리하기 때문이다.

이번에는 다음과 같은 예를 살펴보자.

```
>>> a = [1, 2, 3]
>>> ia = iter(a)
>>> for i in ia:
...     print(i)
...
1
2
3
>>> for i in ia:
...     print(i)
...
>>>
```

이터레이터는 for 문을 이용하여 반복한 후에는 다시 반복해도 값을 가져오지 못한다. 즉, for 문이나 next로 값을 한 번 읽으면 다시 읽을 수 없다. 이터레이터는 현재 위치를 기억하며 앞으로만 진행하기 때문에, 모든 값을 꺼내고 나면 처음으로 되돌아갈 수 없기 때문이다. 다시 반복하려면 `iter()` 함수로 새로운 이터레이터를 만들어야 한다.

이터레이터 만들기

지금까지 iter 함수를 이용하여 리스트를 이터레이터로 변환했다. 이번에는 직접 클래스로 이터레이터를 만들어 보자.

이터레이터 클래스의 필수 메서드

1. `__iter__` 메서드: 이터레이터 객체 자신을 반환한다.
2. `__next__` 메서드: 다음 값을 반환하고, 더 이상 값이 없으면 StopIteration 예외를 발생시킨다.

> *`__iter__` 와 `__next__` 는 05-1에서 배운 `__init__` 처럼 파이썬이 특별한 상황에서 자동으로 호출하는 메서드이다. `iter()` 함수를 호출하면 `__iter__` 가, `next()` 함수를 호출하면 `__next__` 가 자동으로 호출된다.*

이 두 메서드를 구현하면 for 문이나 next() 함수에서 사용할 수 있는 이터레이터가 된다. 다음 예를 살펴보자.

```
# iterator.py
class MyIterator:
    def __init__(self, data):
        self.data = data
        self.position = 0

    def [[MARK]]__iter__[[/MARK]](self):
        return self

    def [[MARK]]__next__[[/MARK]](self):
        if self.position >= len(self.data):
            raise StopIteration
        result = self.data[self.position]
        self.position += 1
        return result

if __name__ == "__main__":
    i = MyIterator([1,2,3])
    for item in i:
        print(item)
```

MyIterator 클래스의 동작 원리를 살펴보자.

1. `__init__` **메서드:** 이터레이터를 초기화한다.
   - `self.data` : 반복할 데이터를 저장한다.
   - `self.position` : 현재 위치를 추적하는 변수이다(0부터 시작).
2. `__iter__` **메서드:** 이터레이터 객체 자신을 반환한다.
   - 이 메서드가 있어야 파이썬이 해당 객체를 반복 가능한 객체로 인식한다.
   - for 문, iter() 함수, next() 함수 등에서 사용하려면 반드시 구현해야 한다.
   - 보통 `return self` 로 자기 자신을 반환한다.
3. `__next__` **메서드:** 다음 값을 반환한다.
   - `self.position` 을 이용해 현재 위치의 값을 가져온다.
   - 위치를 하나씩 증가시킨다.
   - 더 이상 값이 없으면 StopIteration 예외를 발생시킨다.

이 코드를 실행하면 다음과 같은 결과를 확인할 수 있다.

```
1
2
3
```

이번에는 입력받은 데이터를 역순으로 출력하는 ReverseIterator 클래스를 만들어 보자(설명은 앞의 MyIterator와 동일하므로 생략한다).

```
# reviterator.py
class ReverseIterator:
    def __init__(self, data):
        self.data = data
        self.position = len(self.data) -1

    def __iter__(self):
        return self

    def __next__(self):
        if self.position < 0:
            raise StopIteration
        result = self.data[self.position]
        self.position -= 1
        return result

if __name__ == "__main__":
    i = ReverseIterator([1,2,3])
    for item in i:
        print(item)
```

이를 실행하면 다음과 같이 입력받은 데이터를 역순으로 출력한다.

```
3
2
1
```

제너레이터란?

제너레이터는 이터레이터를 쉽게 만들어 주는 함수이다.

이터레이터를 클래스로 만들려면 `__iter__`와 `__next__` 메서드를 구현해야 한다. 하지만 제너레이터를 사용하면 함수 하나로 간단하게 이터레이터를 만들 수 있다.

점프 투 파이썬

제너레이터의 핵심 특징

- 일반 함수와 비슷하지만 `return` 대신 `yield` 키워드를 사용한다.
- `yield`를 만나면 값을 반환하고 함수 실행을 일시 정지한다.
- 다시 호출하면 일시 정지했던 지점부터 계속 실행한다.
- 마치 음악 플레이어의 재생/일시 정지 기능처럼 동작한다.

가장 간단한 제너레이터의 예를 살펴보자.

```
>>> def mygen():
...     [[MARK]]yield[[/MARK]] 'a'
...     [[MARK]]yield[[/MARK]] 'b'
...     [[MARK]]yield[[/MARK]] 'c'
...
>>> g = mygen()
```

mygen 함수는 yield 구문을 포함하므로 제너레이터이다. 제너레이터 객체는 `g = mygen()`과 같이 제너레이터 함수를 호출하여 만들 수 있다. type 함수로 확인하면 g 객체가 제너레이터 타입이라는 것을 알 수 있다.

```
>>> type(g)
<class 'generator'>
```

이제 다음과 같이 제너레이터의 값을 차례대로 얻어 보자.

```
>>> next(g)
'a'
```

제너레이터 객체 g로 next 함수를 실행하면 mygen 함수의 첫 번째 yield 문에서 'a' 값을 반환한다. 제너레이터는 yield를 만나면 값을 반환하고 현재 상태를 그대로 기억한다. 마치 음악을 재생하다가 일시 정지 버튼으로 멈춘 것과 같다.

다시 next() 함수를 실행해 보자.

```
>>> next(g)
'b'
```

이번에는 두 번째 yield 문에서 'b' 값을 반환한다. 계속해서 next 함수를 호출하면 다음과 같은 결과가 출력된다.

```
>>> next(g)
'c'
>>> next(g)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
StopIteration
```

mygen 함수에는 총 3개의 yield 문이 있으므로 네 번째 next를 호출하면 더 이상 반환할 값이 없어 StopIteration 예외가 발생한다.

제너레이터 표현식

이번에는 다음과 같은 예를 살펴보자.

```
# generator.py
def mygen():
    for i in range(1, 1000):
        result = i * i
        yield result

gen = mygen()

print(next(gen))
print(next(gen))
print(next(gen))
```

mygen 함수는 1부터 1,000까지 각각의 숫자를 제곱한 값을 순서대로 반환하는 제너레이터이다. 이 예제를 실행하면 총 3번의 next를 호출하므로 다음과 같은 결과가 나온다.

```
1
4
9
```

제너레이터는 yield를 이용한 함수로 만들 수 있지만, 다음과 같이 소괄호를 사용한 제너레이터 표현식으로 더 간단하게 만들 수도 있다.

```
gen = (i * i for i in range(1, 1000))
```

이 표현식은 mygen 함수로 만든 제너레이터와 똑같이 동작한다. 리스트 컴프리헨션 구문과 비슷하지만, 대괄호 `[]` 대신 소괄호 `()` 를 사용한다. 이를 '제너레이터 표현식(generator expression)'이라고 한다. 소괄호를 사용하지만 튜플을 만드는 것이 아니라 제너레이터 객체를 만든다는 점에 주의하자.

클래스 이터레이터와 제너레이터

클래스를 이용해 이터레이터를 작성하면 복잡한 동작을 구현할 수 있다. 반면 제너레이터를 이용하면 간단하게 이터레이터를 만들 수 있다. 따라서 이터레이터의 성격에 따라 클래스로 만들 것인지, 제너레이터로 만들 것인지 선택해야 한다.

간단한 경우라면 제너레이터 함수나 제너레이터 표현식을 사용하는 것이 가독성이나 유지 보수 측면에서 유리하다. 다음은 `(i * i for i in range(1, 1000))` 제너레이터를 클래스로 구현한 예이다.

```
class MyIterator:
    def __init__(self):
        self.data = 1

    def __iter__(self):
        return self

    def __next__(self):
        if self.data >= 1000:
            raise StopIteration
        result = self.data * self.data
        self.data += 1
        return result
```

이렇게 간단한 경우라면 클래스보다는 제너레이터 표현식을 사용하는 것이 훨씬 간편하고 이해하기 쉽다.

제너레이터 활용하기

제너레이터가 실제로 어떤 도움이 되는지 구체적인 예제로 알아보자.

> *상황: 시간이 오래 걸리는 작업을 처리해야 하는데, 모든 결과가 필요하지 않은 경우*

먼저 일반적인 리스트 컴프리헨션을 사용한 예제를 보자.

```
# generator2.py
import time

def longtime_job():
    print("job start")
    time.sleep(1)  # 1초 지연 - 실제로는 데이터베이스 조회, 파일 처리 등을 시뮬레이션
    return "done"

# 리스트 컴프리헨션: 5번의 작업을 모두 실행해서 리스트로 만든다
list_job = [longtime_job() for i in range(5)]
print(list_job[0])  # 첫 번째 결과만 필요한 상황
```

실행하면 다음과 같은 결과를 출력한다.

```
job start
job start
job start
job start
job start
done
```

총 5초가 걸리고, "job start"가 5번 출력된 후 "done"이 출력된다.

이 코드의 문제점을 살펴보자. `longtime_job()` 함수는 1초씩 걸린다. 리스트를 만들 때 5번의 함수를 모두 미리 실행한다. 첫 번째 결과만 필요한데도 5초를 기다려야 한다.

이제 같은 작업을 제너레이터로 변경해 보자.

```
# generator2.py
import time

def longtime_job():
    print("job start")
    time.sleep(1)
    return "done"

# 제너레이터 표현식: 함수를 미리 실행하지 않고 필요할 때만 실행
[[MARK]]gen_job = (longtime_job() for i in range(5))[[/MARK]]
print(next(gen_job))  # 첫 번째 값만 요청
```

실행 결과는 다음과 같다.

```
job start
done
```

수행 시간도 1초로 단축되었다.

핵심 차이점:

- `[ ]` (리스트 컴프리헨션)을 `( )` (제너레이터 표현식)으로 변경했다.
- `list_job[0]` 을 `next(gen_job)` 으로 변경하여 값을 가져왔다.

제너레이터의 동작 방식:

`(longtime_job() for i in range(5))` 는 코드를 즉시 실행하지 않는다. `next(gen_job)` 을 호출할 때 비로소 첫 번째 `longtime_job()` 만 실행한다. 나머지 4개는 실행되지 않는다.

제너레이터를 사용하면 성능이 크게 개선된다. 실행 시간이 5초에서 1초로 단축되었다. 리스트 컴프리헨션에서는 5개의 함수를 모두 실행했지만, 제너레이터 표현식에서는 필요한 첫 번째 함수만 실행했기 때문이다. "job start"가 5번 출력되던 것이 1번만 출력되는 것을 보면 함수 호출 횟수가 줄어든 것을 확인할 수 있다. 메모리 사용량도 절약되는데, 리스트는 모든 결과를 메모리에 저장하지만 제너레이터는 필요할 때만 값을 생성하기 때문이다.

이런 방식을 '**느긋한 계산법(lazy evaluation)**'이라고 한다. 미리 모든 값을 만들어 두지 않고 필요한 시점에 비로소 값을 계산한다는 뜻이다.

제너레이터는 다음과 같은 상황에서 매우 유용하다.

- 대용량 데이터 처리 시 메모리 절약
- 시간이 오래 걸리는 작업을 필요할 때만 실행
- 무한한 데이터 스트림 처리

07-4 파이썬 타입 어노테이션

파이썬 3.5 버전부터 변수와 함수에 타입을 지정할 수 있는 타입 어노테이션 기능이 추가되었다.

- 동적 언어와 정적 언어
  - 동적 언어의 장단점
- 파이썬 타입 어노테이션
- typing 모듈과 내장 타입의 발전
- mypy

동적 언어와 정적 언어

a 변수에 숫자 1을 대입하고 type 함수를 실행해 보자.

```
>>> a = 1
>>> type(a)
<class 'int'>
```

a 변수의 타입은 int형이다. 이번에는 a 변수에 문자열 "1"을 대입하고 type 함수를 실행해 보자.

```
>>> a = "1"
>>> type(a)
<class 'str'>
```

a 변수의 타입이 str형으로 바뀌었다. 이처럼 프로그램 실행 중에 변수의 타입을 동적으로 바꿀 수 있어 파이썬을 동적 타입 언어(dynamically typed language)라고 한다.

반면 정적 언어에서는 어떨까? 다음은 자바라는 언어의 예인데, 자바를 몰라도 괜찮다. 핵심만 이해하면 된다. 자바에서는 정수형(int) 변수 a에 숫자 1을 대입한 뒤 문자열 "1"을 대입하려 하면 컴파일 오류가 발생한다.

```
int a = 1;  // a 변수를 int형으로 지정
a = "1";  // a 변수에 문자열을 대입할 수 없으므로 컴파일 에러
```

자바는 한 번 변수에 타입을 지정하면 지정한 타입 외에 다른 타입은 사용할 수 없으므로 정적 타입 언어(statically typed language)라고 한다.

동적 언어의 장단점

파이썬과 같은 동적 언어는 타입에 자유로워 유연한 코딩이 가능하고, 쉽고 빠르게 프로그램을 만들 수 있다. 또한 타입 체크를 위한 코드가 없어 비교적 깔끔한 소스 코드를 작성할 수 있다. 하지만 프로젝트 규모가 커질수록 타입을 잘못 사용하여 버그가 생길 확률도 높아진다.

> *안전성을 선호하는 금융권 프로젝트에서는 이런 이유로 동적 언어보다 정적 언어를 주요 언어로 선택하는 경향이 있다.*

파이썬 타입 어노테이션

파이썬은 동적 언어의 단점을 극복하기 위해 3.5 버전부터 타입 어노테이션 기능을 지원하기 시작했다. 다만 정적 언어처럼 강제로 타입을 체크하지 않고, 타입 어노테이션(type annotation) 즉 타입에 대한 힌트를 제공하는 정도의 기능만 지원한다. 이는 동적 언어의 장점을 잃지 않으면서 기존 코드와의 호환성을 유지하려는 선택이다.

> *타입 어노테이션은 코드를 읽는 사람을 위한 힌트이지, 파이썬이 강제로 검사하는 것이 아니다. 타입과 다른 값을 넣어도 프로그램은 정상 동작한다. 타입 검사를 원한다면 mypy 같은 별도 도구를 사용해야 하는데, 이에 대해서는 뒤에서 알아본다.*

타입 어노테이션은 다음과 같이 사용한다.

변수에 타입 지정하기:

```
num: int = 1
name: str = "홍길동"
numbers: list = [1, 2, 3]
```

변수 이름 바로 뒤에 `: int` 처럼 콜론과 타입을 작성하여 num 변수가 int형임을 명시한다. 이는 코드를 읽는 사람에게 해당 변수가 어떤 타입의 값을 저장하는지 미리 알려주는 역할을 한다.

함수에 타입 지정하기:

```
def add(a: int, b: int) -> int:
    return a + b

def greet(name: str) -> str:
    return f"안녕하세요, {name}님!"

def get_user_info(user_id: int) -> dict:
    return {"id": user_id, "name": "홍길동"}
```

함수의 매개변수에도 같은 규칙을 적용하여 타입을 명시할 수 있다. 그리고 `-> int`처럼 화살표 기호를 사용해 함수의 반환값 타입도 명시할 수 있다. 이렇게 하면 함수를 사용하는 사람이 어떤 타입의 인수를 넘겨야 하고, 어떤 타입의 값을 반환받는지 쉽게 알 수 있다.

> *기본 어노테이션 타입으로 정수는 int, 문자열은 str, 리스트는 list, 튜플은 tuple, 딕셔너리는 dict, 집합은 set, 불은 bool을 사용한다.*

리스트나 딕셔너리 같은 타입은 그 안에 어떤 타입의 요소가 들어가는지까지 명시할 수 있다. 대괄호 `[]` 안에 요소의 타입을 적으면 된다.

```
numbers: list[int] = [1, 2, 3]               # int를 담는 리스트
user_info: dict[str, int] = {"age": 30}       # 키는 str, 값은 int인 딕셔너리
coordinates: tuple[float, float] = (3.5, 7.2) # float 2개를 담는 튜플
```

> *`list[int]` 처럼 대괄호 안에 타입을 넣는 문법은 파이썬 3.9 이상에서 사용할 수 있다.*

typing 모듈과 내장 타입의 발전

앞서 본 기본 타입 어노테이션(`int` , `str` , `list` 등)만으로도 많은 경우를 처리할 수 있지만, 더 구체적인 타입 정보가 필요할 때가 있다. 이때 사용하는 것이 `typing` 모듈이다.

파이썬 3.9 이상을 사용한다면 소문자 내장 타입(`list` , `dict` , `tuple`)을 그대로 사용하면 된다. 하지만 과거 코드에서는 `typing` 모듈의 대문자 타입(`List` , `Dict` , `Tuple`)을 사용하는 경우가 많으므로 두 가지 방식을 모두 알아 두자.

파이썬 버전별 타입 어노테이션의 발전:

```
# 파이썬 3.5~3.8: typing 모듈 필수
from typing import List, Dict, Tuple, Optional

numbers: List[int] = [1, 2, 3]
user_info: Dict[str, int] = {"age": 30}

# 파이썬 3.9 이상: 내장 타입 사용 가능 (권장)
numbers: list[int] = [1, 2, 3]
user_info: dict[str, int] = {"age": 30}
```

파이썬 3.9부터는 내장 타입을 직접 사용할 수 있어 `typing` 모듈의 `List`, `Dict`, `Tuple`을 가져올 필요가 없어졌다. 하지만 여전히 `typing` 모듈이 필요한 경우가 있다.

typing 모듈이 여전히 필요한 경우:

```
from typing import Optional, Union

# 1. Optional - None이 가능한 경우
user_name: Optional[str] = None  # str 또는 None

# 2. Union - 여러 타입이 가능한 경우
user_id: Union[int, str] = "jenny"  # 정수 또는 문자열
```

`Optional[str]`은 값이 str일 수도 있고 None일 수도 있다는 의미이다. `Union[int, str]`은 값이 int 또는 str 중 하나라는 의미이다. 이 두 가지가 실무에서 가장 많이 사용된다.

점프 투 파이썬

typing 모듈의 고급 타입

typing 모듈에는 `Optional`과 `Union` 외에도 다양한 타입이 있다.

```
from typing import Callable, Any

# Callable - 함수 타입 지정
def process_data(callback: Callable[[int], str]) -> str:
    return callback(42)

# Any - 모든 타입 허용
unknown_data: Any = {"key": "value"}
```

`Callable[[int], str]`은 int를 인수로 받아 str을 반환하는 함수를 의미한다. `Any`는 어떤 타입이든 허용한다는 의미이다.

실무에서의 권장사항:

```
# 파이썬 3.9 이상을 사용한다면
from typing import Optional, Union  # 필요한 것만 가져오기

# 기본 타입은 내장 타입 사용
scores: list[int] = [95, 87, 92]
user_data: dict[str, str] = {"name": "홍길동"}

# 특별한 경우에만 typing 모듈 사용
def find_user(user_id: int) -> Optional[dict[str, str]]:
    # 사용자를 찾으면 딕셔너리 반환, 없으면 None 반환
    if user_id > 0:
        return {"name": "홍길동", "email": "hong@example.com"}
    return None
```

하위 호환성 고려:

파이썬 3.8 이하 버전을 지원해야 한다면 여전히 `typing` 모듈을 사용해야 한다. 많은 기업 환경에서는 안정성을 위해 구버전을 사용하는 경우가 많아 `typing` 모듈에 대한 이해는 여전히 중요하다.

mypy

파이썬은 타입 어노테이션으로 매개변수의 타입을 명시하더라도 다음과 같이 다른 타입의 인수를 입력할 수 있다.

```
# c:/doit/typing_sample.py
def add(a: int, b: int) -> int:
    return a+b

result = add(3, [[MARK]]3.4[[/MARK]])
print(result)  # 6.4 출력
```

add 함수의 b 매개변수는 int형으로 지정했지만, 3.4와 같은 float형 데이터를 사용해도 이 코드는 문제 없이 실행된다. 파이썬 타입 어노테이션이 강제적인 타입 체크가 아닌 단순한 힌트 역할만 하기 때문이다. 즉, 타입 어노테이션을 무시해도 프로그램은 정상 동작한다.

> *파이참과 같은 파이썬 전용 IDE를 사용하면 타입이 맞지 않는다는 경고 메시지를 표시한다.*

파이썬 자체로는 타입 어노테이션을 검사하지 않지만, 별도 도구를 사용하면 더 적극적으로 타입을 검사할 수 있다. 대표적인 도구가 mypy다. mypy는 정적 타입 검사기로, 코드를 실행하기 전에 타입 어노테이션이 올바른지 미리 확인해 준다. mypy는 파이썬 표준 라이브러리가 아니므로 다음과 같이 설치해야 한다.

```
c:\doit> pip install mypy
```

맥/리눅스에서는 다음처럼 설치한다.

```
$ pip install mypy
```

mypy 설치 후 다음과 같이 사용해 보자.

```
C:\doit>mypy typing_sample.py
typing_sample.py:5: error: Argument 2 to "add" has incompatible type "float"; expected "int"
Found 1 error in 1 file (checked 1 source file)
```

mypy로 typing_sample.py 파일을 확인하면 타입이 맞지 않는다는 오류가 발생한다. 다음과 같이 코드를 수정해 보자.

```
def add(a: int, b: int) -> int:
    return a+b

result = add(3, 4)
print(result)
```

오류가 발생했던 3.4를 int형에 맞게 4로 변경했다. mypy를 다시 실행해 보면 오류가 없다는 것을 알려 준다.

```
c:\doit>mypy typing_sample.py
Success: no issues found in 1 source file
```

파이썬 타입 어노테이션은 최근 사용이 급속도로 늘어나고 있다. 특히 대규모 프로젝트나 팀 개발에서는 코드의 가독성과 유지보수성을 높이기 위해 타입 어노테이션을 적극 활용한다. FastAPI, Django 등 인기 있는 프레임워크들도 타입 어노테이션을 적극 지원하므로, 현대 파이썬 개발에서 필수 기능이라고 할 수 있다.